

Problem A. Distinct Numbers

Time Limit 1000 ms

Mem Limit 524288 kB

You are given a list of n integers, and your task is to calculate the number of *distinct* values in the list.

Input

The first input line has an integer n : the number of values.

The second line has n integers $x_1, x_2, \dots, x_n$.

Output

Print one integers: the number of distinct values.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

Example

Input	Output
5 2 3 2 2 3	2

Problem B. Strictly Increasing?

Time Limit 2000 ms

Problem Statement

You are given a positive integer N and a sequence of positive integers $A = (A_1, A_2, \dots, A_N)$ of length N .

Determine whether A is strictly increasing, that is, whether $A_i < A_{i+1}$ holds for every integer i with $1 \leq i < N$.

Constraints

- $2 \leq N \leq 100$
- $1 \leq A_i \leq 1000$ ($1 \leq i \leq N$)
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N
A1 A2 ... AN
```

Output

If A is strictly increasing, print **Yes**; otherwise, print **No**.

The judge is case-insensitive. For example, if the correct answer is **Yes**, any of **yes**, **YES**, and **yEs** will be accepted.

Sample 1

Input	Output
3 1 2 5	Yes

$A_1 < A_2$ and $A_2 < A_3$, so A is strictly increasing.

Sample 2

Input	Output
3 3 9 5	No

$A_1 < A_2$, but $A_2 < A_3$ does not hold, so A is not strictly increasing.

Sample 3

Input	Output
10 1 1 2 3 5 8 13 21 34 55	No

$A_1 < A_2$ does not hold, so A is not strictly increasing.

Problem C. K Swap

Time Limit 2000 ms

Problem Statement

We have a sequence of length N : $A = (a_1, \dots, a_N)$. Additionally, you are given an integer K .

You can perform the following operation zero or more times.

- Choose an integer i such that $1 \leq i \leq N - K$, then swap the values of a_i and a_{i+K} .

Determine whether it is possible to sort A in ascending order.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq K \leq N - 1$
- $1 \leq a_i \leq 10^9$
- All values in input are integers.

Input

Input is given from Standard Input in the following format:

```
N K
a_1 ... a_N
```

Output

If it is possible to sort A in ascending order, print **Yes**; otherwise, print **No**.

Sample 1

Input	Output
5 2 3 4 1 3 4	Yes

The following sequence of operations sorts A in ascending order.

- Choose $i = 1$ to swap the values of a_1 and a_3 . A is now $(1, 4, 3, 3, 4)$.
- Choose $i = 2$ to swap the values of a_2 and a_4 . A is now $(1, 3, 3, 4, 4)$.

Sample 2

Input	Output
5 3 3 4 1 3 4	No

Sample 3

Input	Output
7 5 1 2 3 4 5 5 10	Yes

No operations may be needed.

Problem D. Counting

Time Limit 2000 ms

Problem Statement

How many integers not less than A and not more than B are there?

Constraints

- $1 \leq A \leq 100$
- $1 \leq B \leq 100$
- A and B are integers.

Input

Input is given from Standard Input in the following format:

A B

Output

Print the number of integers not less than A and not more than B .

Sample 1

Input	Output
2 4	3

We have three integers not less than 2 and not more than 4: 2, 3, 4, so we should print 3.

Sample 2

Input	Output
10 100	91

Sample 3

Input	Output
3 2	0

We have no integers not less than 3 and not more than 2, so we should print 0.

Problem E. Increasing

Time Limit 1000 ms

Mem Limit 262144 kB

You are given an array a of n positive integers. Determine if, by rearranging the elements, you can make the array strictly increasing. In other words, determine if it is possible to rearrange the elements such that $a_1 < a_2 < \dots < a_n$ holds.

Input

The first line contains a single integer t ($1 \leq t \leq 100$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 100$) — the length of the array.

The second line of each test case contains n integers a_i ($1 \leq a_i \leq 10^9$) — the elements of the array.

Output

For each test case, output "YES" (without quotes) if the array satisfies the condition, and "NO" (without quotes) otherwise.

You can output the answer in any case (for example, the strings "yEs", "yes", "Yes" and "YES" will be recognized as a positive answer).

Examples

Input	Output
3 4 1 1 1 1 5 8 7 1 3 4 1 5	NO YES YES

Note

In the first test case any rearrangement will keep the array $[1, 1, 1, 1]$, which is not strictly increasing.

In the second test case, you can make the array $[1, 3, 4, 7, 8]$.

Problem F. Gravity Flip

Time Limit 1000 ms

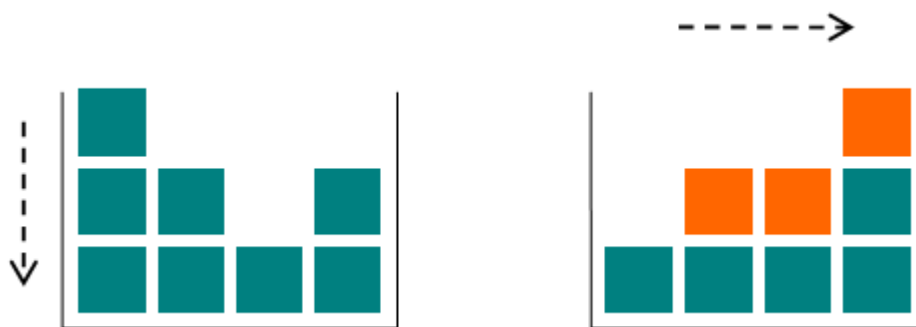
Mem Limit 262144 kB

Input File stdin

Output File stdout

Little Chris is bored during his physics lessons (too easy), so he has built a toy box to keep himself occupied. The box is special, since it has the ability to change gravity.

There are n columns of toy cubes in the box arranged in a line. The i -th column contains a_i cubes. At first, the gravity in the box is pulling the cubes downwards. When Chris switches the gravity, it begins to pull all the cubes to the right side of the box. The figure shows the initial and final configurations of the cubes in the box: the cubes that have changed their position are highlighted with orange.



Given the initial configuration of the toy cubes in the box, find the amounts of cubes in each of the n columns after the gravity switch!

Input

The first line of input contains an integer n ($1 \leq n \leq 100$), the number of the columns in the box. The next line contains n space-separated integer numbers. The i -th number a_i ($1 \leq a_i \leq 100$) denotes the number of cubes in the i -th column.

Output

Output n integer numbers separated by spaces, where the i -th number is the amount of

cubes in the i -th column after the gravity switch.

Examples

Input	Output
4 3 2 1 2	1 2 2 3

Input	Output
3 2 3 8	2 3 8

Note

The first example case is shown on the figure. The top cube of the first column falls to the top of the last column; the top cube of the second column falls to the top of the third column; the middle cube of the first column falls to the top of the second column.

In the second example case the gravity switch does not change the heights of the columns.

Problem G. Balls and Bag Query

Time Limit 2000 ms

Problem Statement

You have an empty bag. You are given Q queries, which must be processed in order.

There are three types of queries.

- **1 x** : Put one ball with the integer x written on it into the bag.
- **2 x** : Remove one ball with the integer x written on it from the bag and discard it. It is guaranteed that the bag has a ball with the integer x written on it when this query is given.
- **3** : Print the number of different integers written on the balls in the bag.

Constraints

- $1 \leq Q \leq 2 \times 10^5$
- $1 \leq x \leq 10^6$
- When a query of the second type is given, the bag has a ball with the integer x written on it.
- There is at least one query of the third type.
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
Q
query1
query2
⋮
queryQ
```

The i -th query query _{i} is given in one of the following three formats:

```
1  $x$ 
```

2 x

3

Output

If there are K queries of the third type, print K lines. The i -th line ($1 \leq i \leq K$) should contain the answer to the i -th query of the third type.

Sample 1

Input	Output
8 1 3 1 1 1 4 3 2 1 3 1 5 3	3 2 3

Initially, the bag is empty.

For the first query **1 3**, a ball with the integer 3 written on it enters the bag.

For the second query **1 1**, a ball with the integer 1 written on it enters the bag.

For the third query **1 4**, a ball with the integer 4 written on it enters the bag.

For the fourth query **3**, the bag has balls with the integers 1, 3, 4, so print 3.

For the fifth query **2 1**, a ball with the integer 1 written on it is removed from the bag.

For the sixth query **3**, the bag has balls with the integers 3, 4, so print 2.

For the seventh query **1 5**, a ball with the integer 5 written on it enters the bag.

For the eighth query **3**, the bag has balls with the integers 3, 4, 5, so print 3.

Sample 2

Input	Output
8 1 2 1 2 3 2 2 1 4 1 4 2 2 3	1 1

Problem H. Max - Min Query

Time Limit 2000 ms

Problem Statement

We have a multiset of integers S , which is initially empty.

Given Q queries, process them in order. Each query is of one of the following types.

- **1 x**: Insert an x into S .
- **2 x c**: Remove an x from S m times, where $m = \min(c, (\text{the number of } x\text{'s contained in } S))$.
- **3**: Print (maximum value of S) – (minimum value of S). It is guaranteed that S is not empty when this query is given.

Constraints

- $1 \leq Q \leq 2 \times 10^5$
- $0 \leq x \leq 10^9$
- $1 \leq c \leq Q$
- When a query of type **3** is given, S is not empty.
- All values in input are integers.

Input

Input is given from Standard Input in the following format:

```
Q
query1
⋮
queryQ
```

query _{i} , which denotes the i -th query, is in one of the following formats:

```
1 x
```

```
2 x c
```

3

Output

Print the answers for the queries of type 3 in the given order, separated by newlines.

Sample 1

Input	Output
8	1
1 3	5
1 2	4
3	
1 2	
1 7	
3	
2 2 3	
3	

The multiset S transitions as follows.

- 1-st query: insert 3 into S . S is now $\{3\}$.
- 2-nd query: insert 2 into S . S is now $\{2, 3\}$.
- 3-rd query: the maximum value of $S = \{2, 3\}$ is 3 and its minimum value is 2, so print $3 - 2 = 1$.
- 4-th query: insert 2 into S . S is now $\{2, 2, 3\}$.
- 5-th query: insert 7 into S . S is now $\{2, 2, 3, 7\}$.
- 6-th query: the maximum value of $S = \{2, 2, 3, 7\}$ is 7 and its minimum value is 2, so print $7 - 2 = 5$.
- 7-th query: since there are two 2's in S and $\min(2, 3) = 2$, remove 2 from S twice. S is now $\{3, 7\}$.
- 8-th query: the maximum value of $S = \{3, 7\}$ is 7 and its minimum value is 3, so print $7 - 3 = 4$.

Sample 2

Input	Output
4	
1 10000	
1 1000	
2 100 3	
1 10	

If the given queries do not contain that of type 3, nothing should be printed.